

RNN

LSTM

GRU

# Recurrent Neural Networks

## RNN, LSTM & GRU

Understanding sequence models — from intuition to the gate equations

*A complete visual guide • 40+ slides*

# What this deck covers



## Why sequences?

Motivation and the limits of feedforward nets

01



## Recurrent Neural Networks

Architecture, unrolling, BPTT & the gradient problem

02



## Long Short-Term Memory

The cell state and its three gates

03



## Gated Recurrent Unit

A lighter gating: update & reset gates

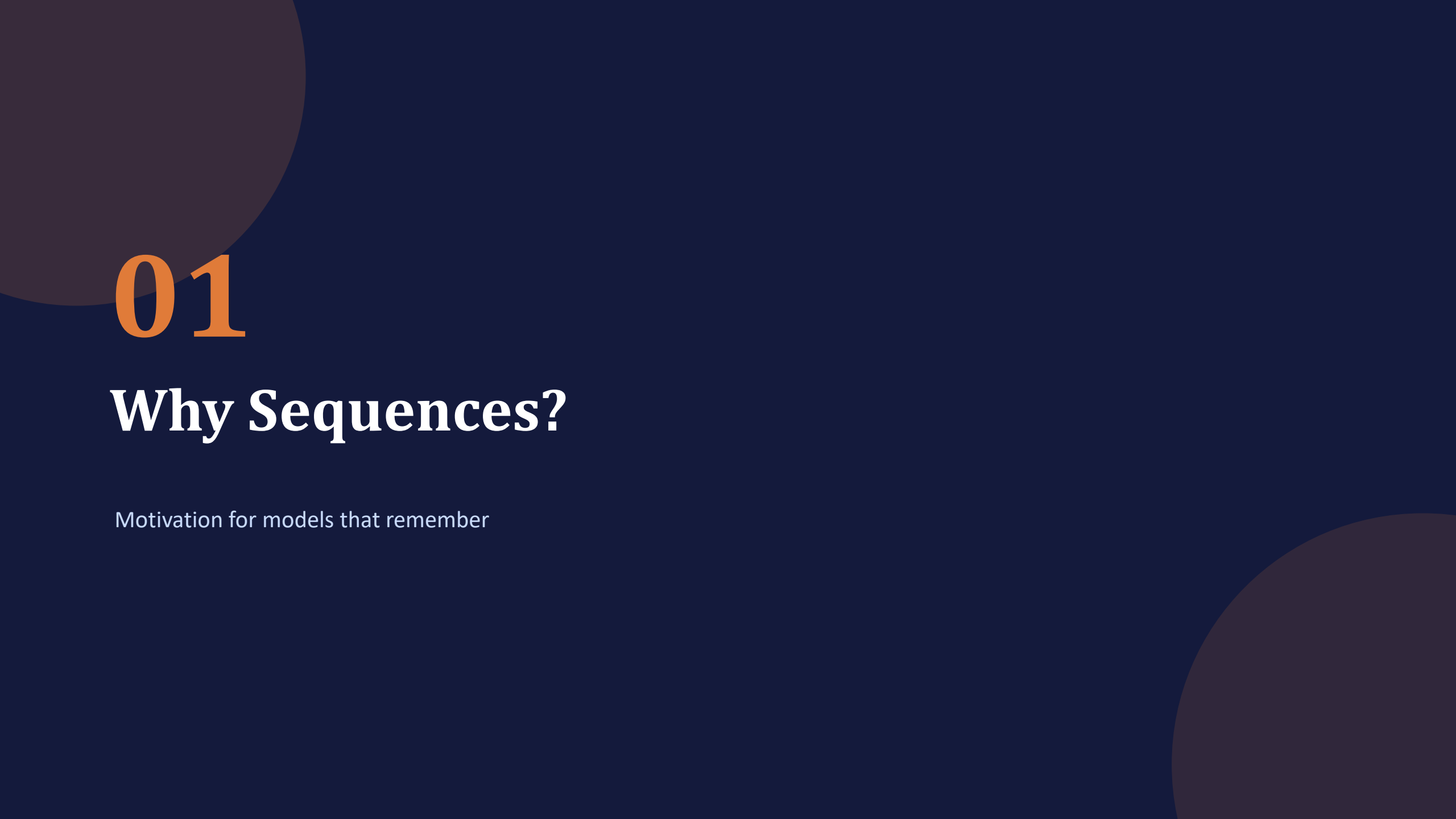
04



## Comparison & applications

When to use which, plus practical tips

05



# 01

## Why Sequences?

Motivation for models that remember

# Sequential data is everywhere



## Language

“The clouds are in the \_\_\_\_” — order and context decide the next word



## Speech & audio

A waveform is a signal that unfolds over time



## Time series

Stock prices, weather, sensor readings, ECG signals



## Video

A sequence of frames where each depends on the last

*Common thread: the order of inputs carries meaning, and each element depends on what came before.*

# Why not a plain feedforward network?

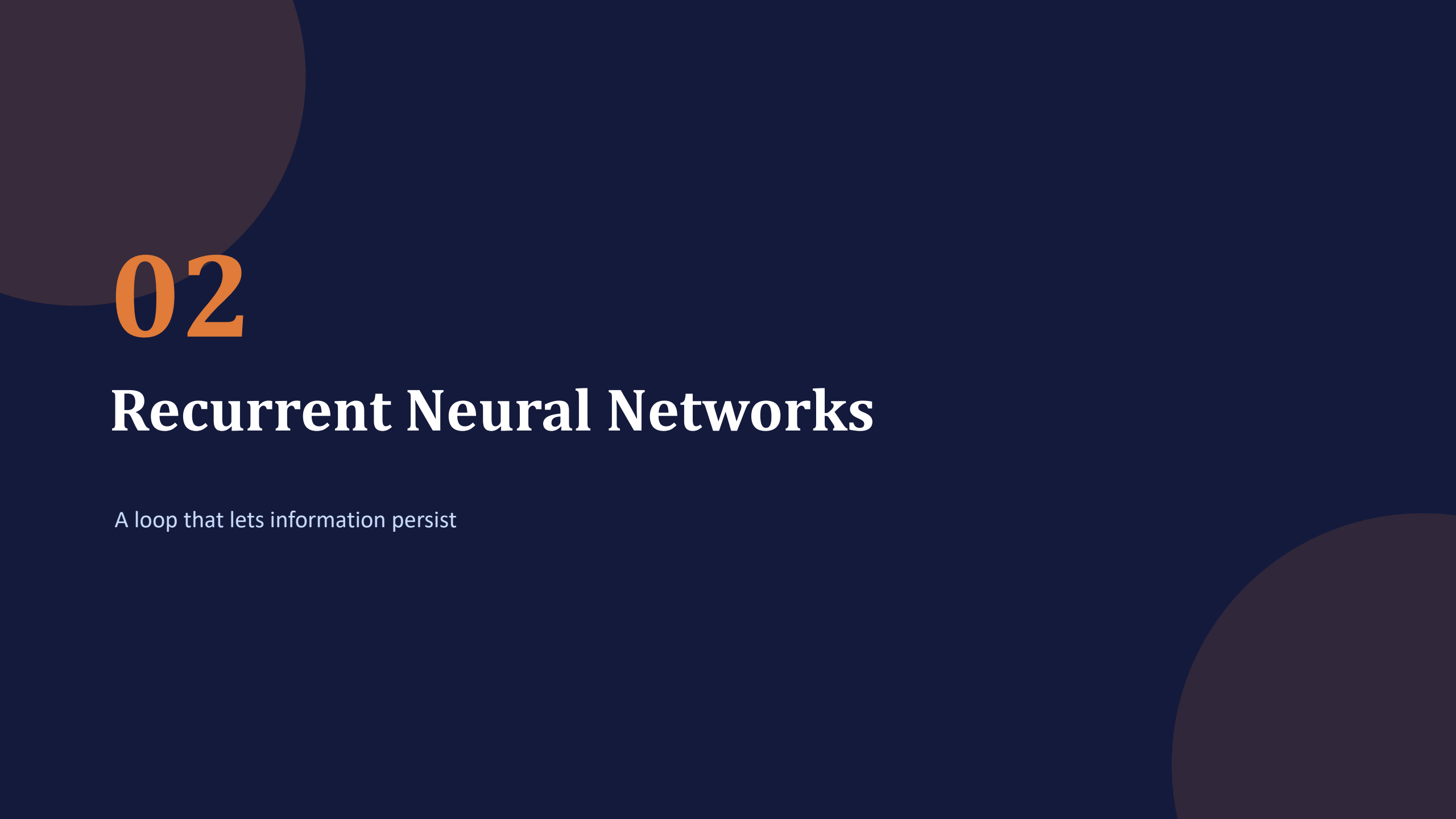
## Feedforward / CNN limits

- Fixed-size input and output — but sequences vary in length
- No memory: each input is processed independently
- Cannot share what it learns across positions in the sequence
- Word order is ignored — “dog bites man” = “man bites dog”

## What we actually need

- Handle variable-length input and output
- A memory that carries context forward in time
- Share the same weights across every time step
- Sensitivity to order and long-range dependencies

→ This is exactly what a Recurrent Neural Network is designed to do.



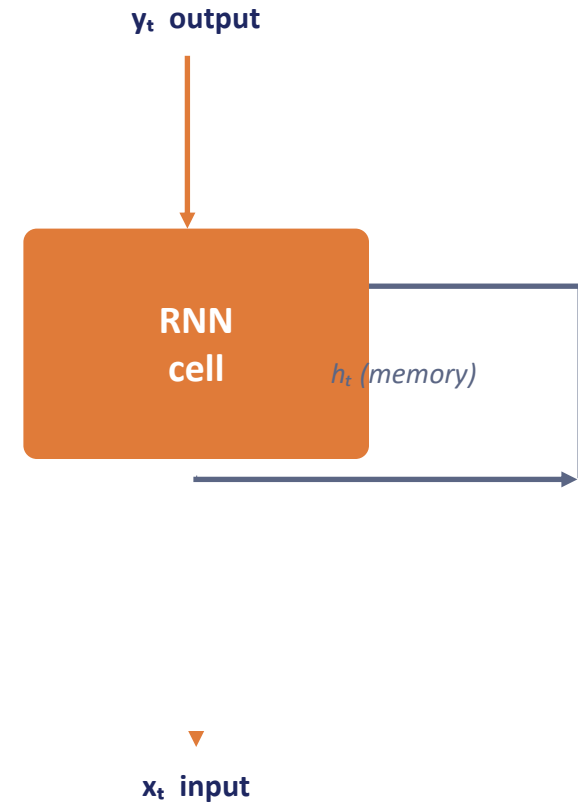
# 02

## Recurrent Neural Networks

A loop that lets information persist

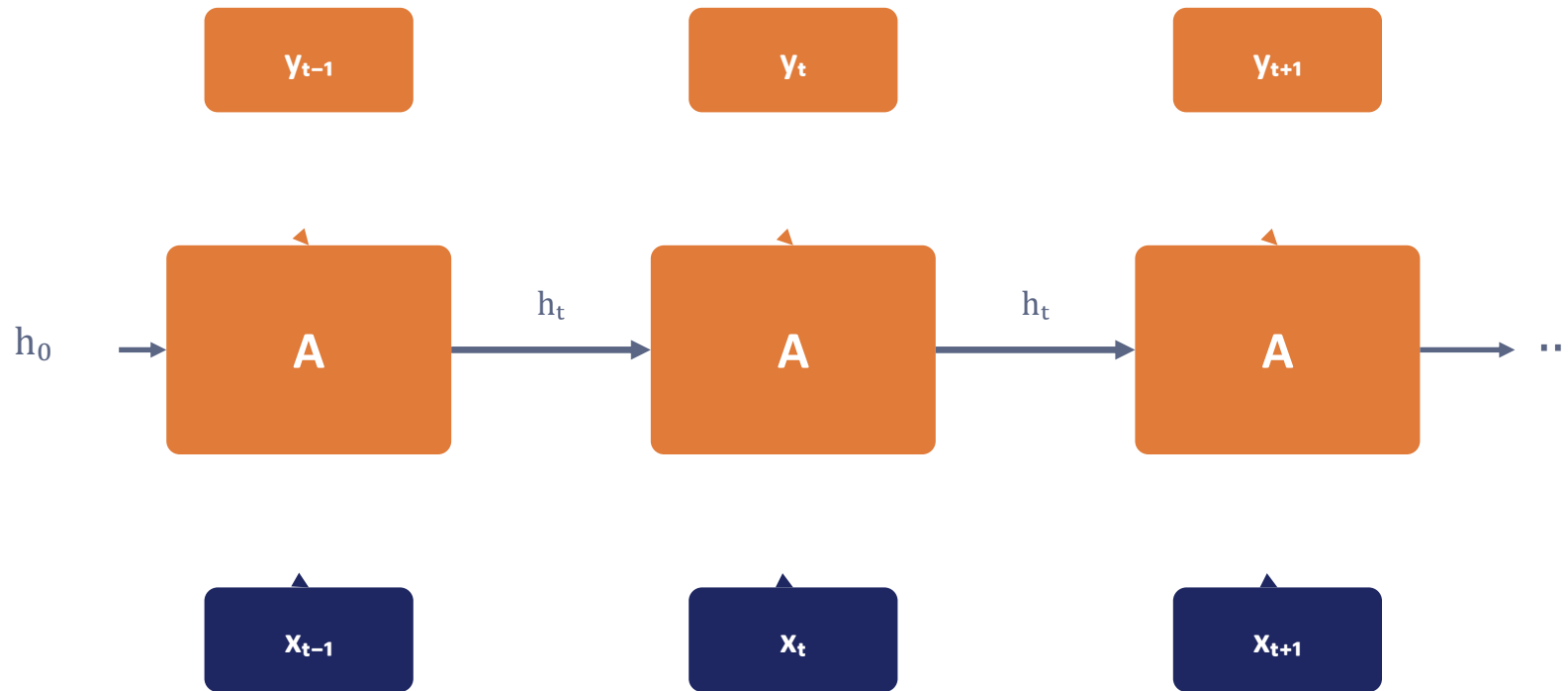
# The core idea: a loop

- An RNN processes a sequence one element at a time.
- At each step it combines the current input with a hidden state — its memory of everything seen so far.
- The same weights are reused at every time step (weight sharing).
- The hidden state is fed back into the network — the “recurrent” loop.



# Unrolling the loop through time

The same cell, drawn once per time step — reveals how information flows left to right.



*Every box "A" shares the same weights. The horizontal arrows carry the hidden state — the network's memory.*



# The math of a vanilla RNN

At each time step  $t$ , two equations are computed:

$$h_t = \tanh( W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t + b_h )$$

*Hidden state: blends previous memory  $h_{t-1}$  with the new input  $x_t$*

$$y_t = W_{hy} \cdot h_t + b_y$$

*Output: a read-out from the current hidden state (a softmax is often applied on top)*

## Key terms

$h_t$  hidden state / memory    $x_t$  input at step  $t$     $W$  shared weight matrices    $\tanh$  squashes values to  $(-1, 1)$

# One framework, many shapes

## One → One

Fixed in, fixed out (plain net)

*e.g. Image classification*

## One → Many

One input, a sequence out

*e.g. Image captioning*

## Many → One

A sequence in, one output

*e.g. Sentiment analysis*

## Many → Many

Sequence in and out

*e.g. Machine translation*

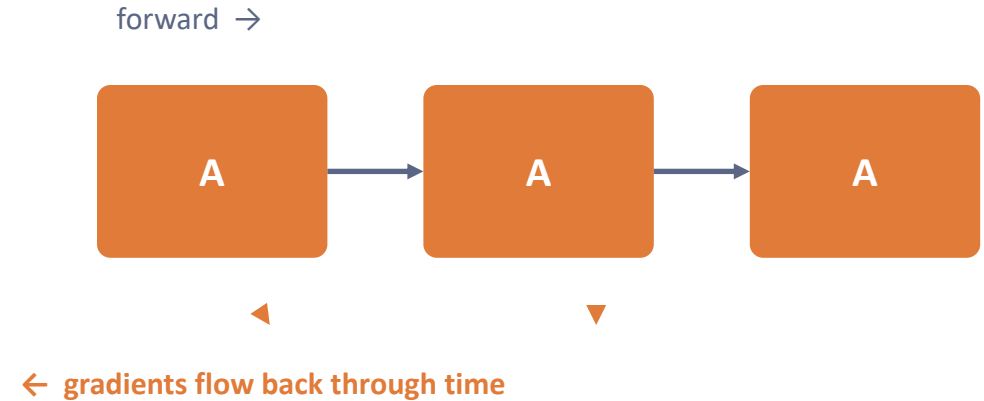
## Sync Many → Many

Aligned sequence labeling

*e.g. Video frame tagging*

# Backpropagation Through Time (BPTT)

- To train an RNN we unroll it across all time steps into one big computational graph.
- The loss at each step is summed, then gradients flow backward through the entire unrolled network.
- Because weights are shared, the gradient for each weight is the sum of its contributions at every time step.
- This is ordinary backprop — just applied to the time-unrolled graph.
- Cost grows with sequence length, so long sequences are often truncated (truncated BPTT).



# Vanishing & exploding gradients

## Vanishing gradients

- Backprop multiplies many small derivatives ( $< 1$ ) together.
- Over many steps the product shrinks toward zero.
- Early time steps receive almost no gradient signal.
- Result: the network cannot learn long-range dependencies.

## Exploding gradients

- If derivatives are  $> 1$ , the product blows up instead.
- Weights receive huge updates and training diverges (NaNs).
- Easier to spot and to fix than vanishing.
- Common remedy: gradient clipping to a maximum norm.

# The long-term dependency gap

*“I grew up in France ... I speak fluent \_\_\_\_”*

To predict “French”, the network must recall a word from many steps earlier. Vanilla RNNs struggle when that gap is large.

- In theory RNNs can carry information for arbitrarily long — in practice, vanishing gradients prevent it.
- Short gaps (a few steps) are fine; long gaps are where vanilla RNNs break down.
- We need an architecture that can preserve information over long distances and decide what to keep or forget.
- → Enter gated units: LSTM and GRU.



03

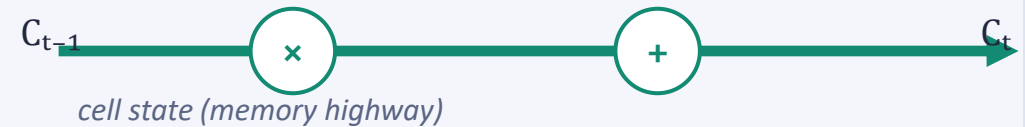
# Long Short-Term Memory

Gates that learn what to remember and forget



# The big idea: a protected memory

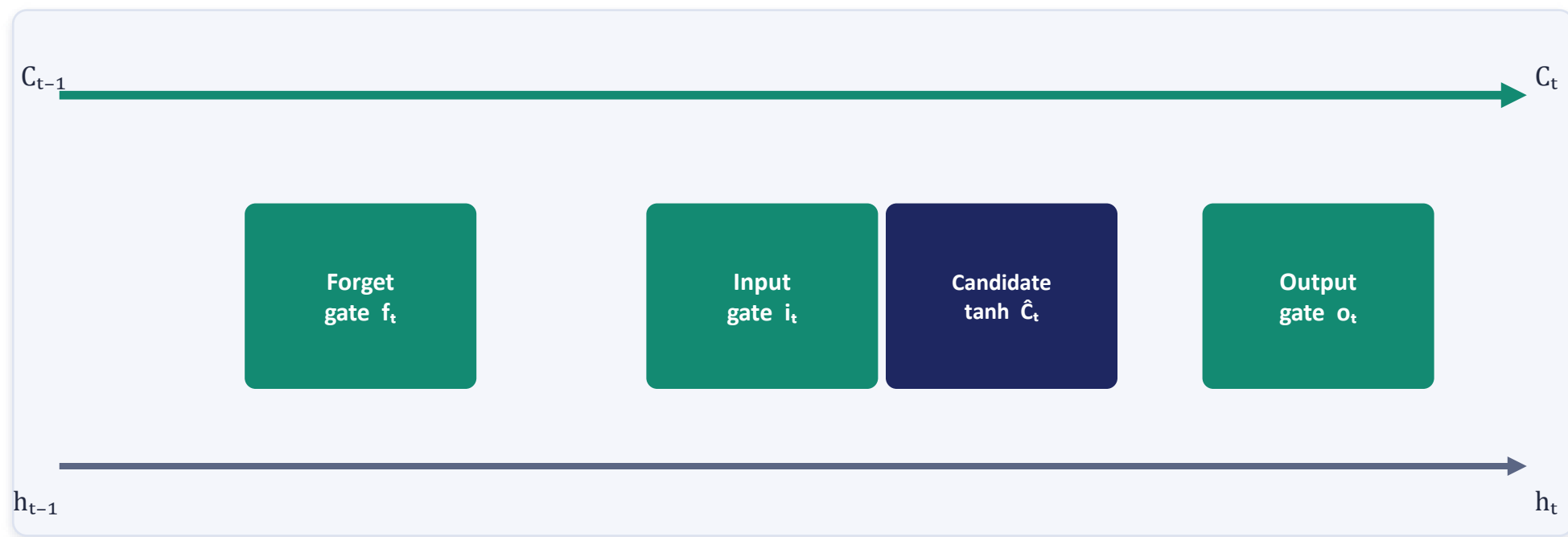
- LSTM (Hochreiter & Schmidhuber, 1997) adds a separate cell state  $C_t$  — a “conveyor belt” of memory.
- Information flows along the cell state with only minor linear interactions, so it is easy to preserve unchanged.
- Three gates regulate that belt: what to forget, what to add, and what to output.
- Gates are small neural layers with a sigmoid that output values in  $(0, 1)$  — a soft “how much to let through”.



gates interact with the belt only through  $\times$  and  $+$

# Anatomy of an LSTM cell

Two things flow between steps: the cell state  $C_t$  (top) and the hidden state  $h_t$  (bottom).

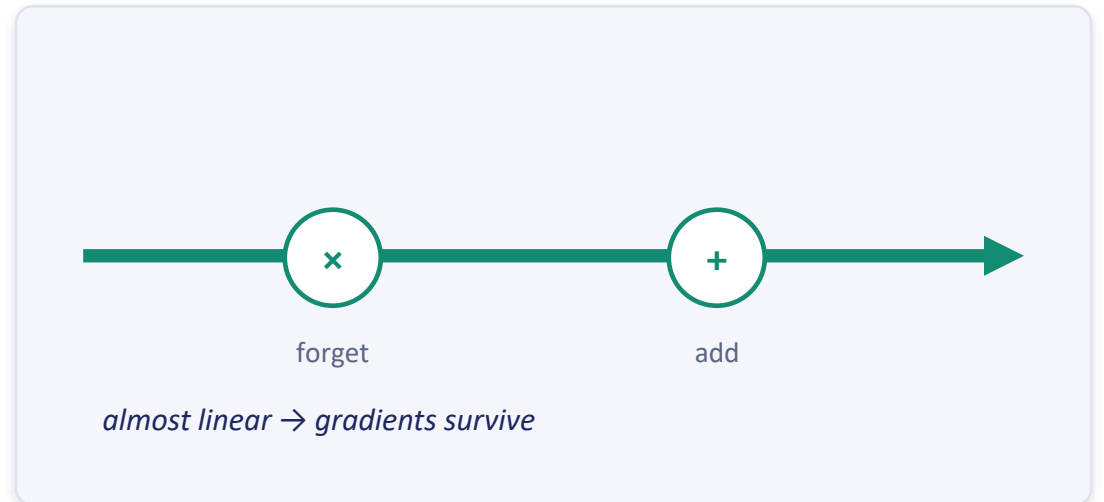


*Sigmoid gates (teal) output 0–1 and act as valves; tanh layers propose new content.*



# The cell state: the memory highway

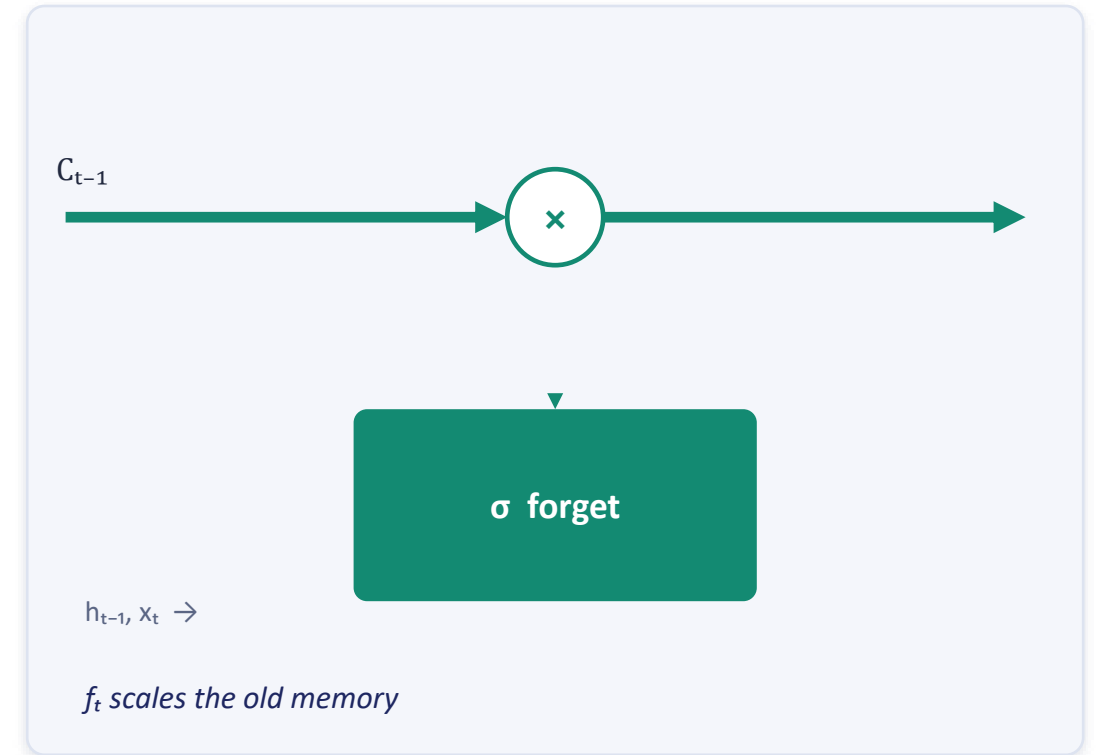
- The cell state  $C_t$  runs straight through the whole chain.
- It is changed only by two controlled operations: a  $\times$  (forget) and a  $+$  (add).
- Because there is no repeated matrix multiplication or squashing along this path, gradients can flow far without vanishing.
- This uninterrupted path is the key to learning long-term dependencies.



# Forget gate — what to throw away

- Looks at the previous hidden state  $h_{t-1}$  and current input  $x_t$ .
- A sigmoid outputs a number in  $(0, 1)$  for each value in the cell state.
- 1 = “keep this completely”, 0 = “erase this completely”.
- Example: on seeing a new subject, forget the gender of the old subject.

$$f_t = \sigma( Wf \cdot [h_{t-1}, x_t] + bf )$$



# Input gate — what new info to store

- Two parts work together to decide the update:
- The input gate  $i_t$  (sigmoid) decides which values to update.
- A tanh layer creates candidate values  $\hat{C}_t$  — new content that could be added.
- Their product  $i_t \odot \hat{C}_t$  is the proposed addition to memory.

$$i_t = \sigma( W_i \cdot [h_{t-1}, x_t] + b_i )$$

$$\hat{C}_t = \tanh( W_C \cdot [h_{t-1}, x_t] + b_C )$$

$$i_t \odot \hat{C}_t = \text{new candidate memory}$$



$\sigma$  input  $i_t$

$\tanh \hat{C}_t$

# Update the cell state

Now combine the two decisions — forget old, add new:

$$C_t = f_t \odot C_{t-1} + i_t \odot \hat{C}_t$$

$$f_t \odot C_{t-1}$$

Forget: scale down the parts of old memory we no longer need.

$$i_t \odot \hat{C}_t$$

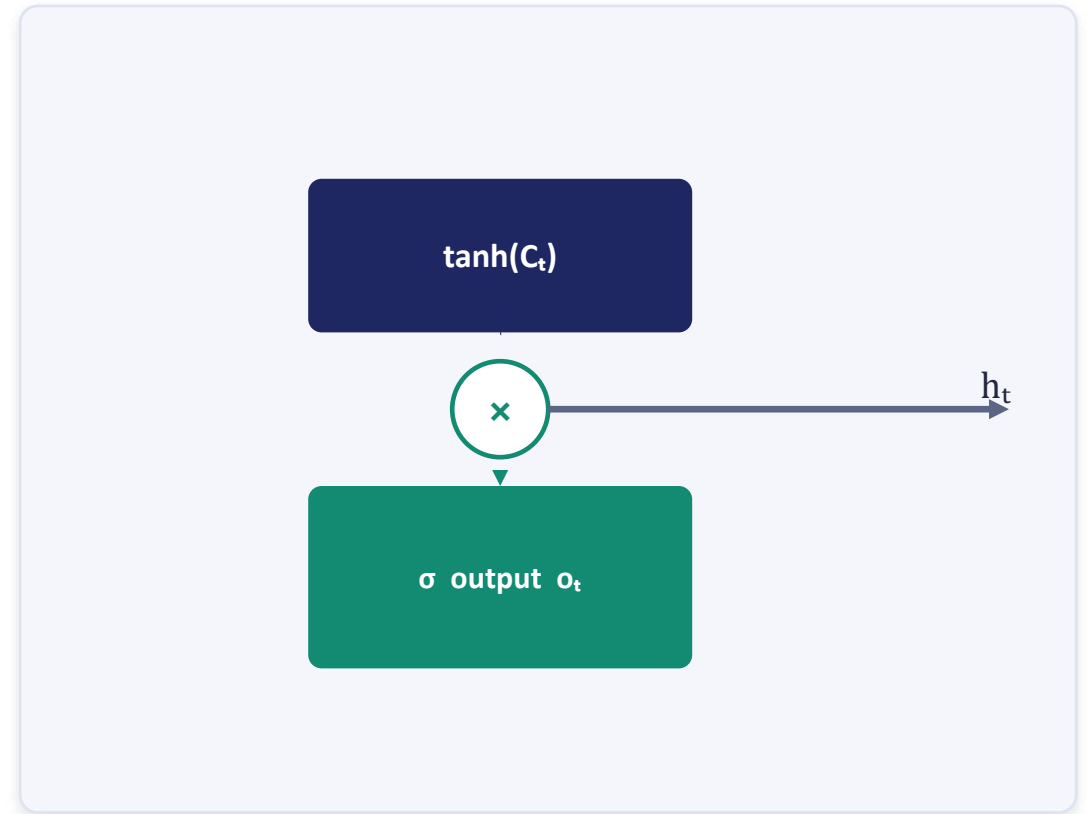
Add: write the new candidate values, scaled by how much we want them.

# Output gate — what to reveal

- The hidden state  $h_t$  is a filtered view of the cell state.
- The output gate  $o_t$  (sigmoid) decides which parts of the memory to expose.
- The cell state is passed through  $\tanh$ , then scaled by  $o_t$ .
- $h_t$  is what the next step sees and what produces the output  $y_t$ .

$$o_t = \sigma( W_o \cdot [h_{t-1}, x_t] + b_o )$$

$$h_t = o_t \odot \tanh( C_t )$$



# All six equations together

## FORGET GATE

$$f_t = \sigma( Wf \cdot [h_{t-1}, x_t] + bf )$$

## INPUT GATE

$$i_t = \sigma( Wi \cdot [h_{t-1}, x_t] + bi )$$

## CANDIDATE MEMORY

$$\hat{C}_t = \tanh( WC \cdot [h_{t-1}, x_t] + bC )$$

## UPDATE CELL STATE

$$C_t = f_t \odot C_{t-1} + i_t \odot \hat{C}_t$$

## OUTPUT GATE

$$o_t = \sigma( Wo \cdot [h_{t-1}, x_t] + bo )$$

## HIDDEN STATE

$$h_t = o_t \odot \tanh( C_t )$$

# How LSTM defeats vanishing gradients

- The cell-state update  $C_t = f_t \odot C_{t-1} + i_t \odot \hat{C}_t$  is additive, not a repeated multiplication by a weight matrix.
- When the forget gate stays near 1, the gradient of  $C_t$  with respect to  $C_{t-1}$  is  $\approx 1$  — so error signals pass backward almost unchanged.
- This “constant error carousel” lets gradients travel across hundreds of steps without shrinking to zero.
- The gates are learned, so the network decides for itself how long to hold each piece of information.
- Trade-off: four times as many parameters as a vanilla RNN, and more computation per step.



# 04

## Gated Recurrent Unit

The same idea, streamlined



# A simpler gated unit

- GRU (Cho et al., 2014) simplifies the LSTM.
- It merges the cell state and hidden state into a single  $h_t$ .
- It uses just two gates instead of three: an update gate and a reset gate.
- Fewer parameters  $\rightarrow$  faster to train and often comparable in accuracy.
- A popular default when you want LSTM-like power with less overhead.

## Two gates only

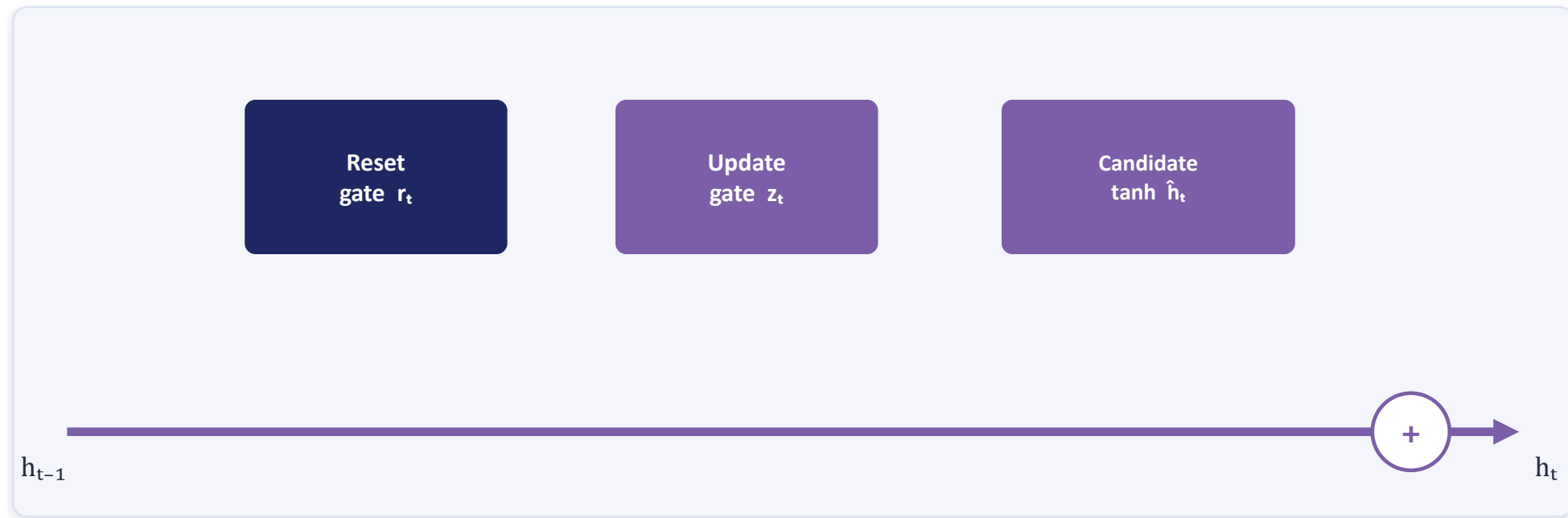
Update gate  $z_t$  — how much of the past to keep

Reset gate  $r_t$  — how much past to ignore now

*No separate cell state, no output gate.*

# Anatomy of a GRU cell

Only the hidden state  $h_t$  flows between steps — no separate memory line.



*The update gate does the job of LSTM's forget + input gates at once.*

# Update gate — keep or refresh

- Decides how much of the previous hidden state to carry forward unchanged.
- $z_t$  close to 1  $\rightarrow$  keep the old state (like LSTM remembering).
- $z_t$  close to 0  $\rightarrow$  replace it with new content.
- It controls the balance between past and present in the final blend.

$$z_t = \sigma( Wz \cdot [h_{t-1}, x_t] + bz )$$

## Interpolation view

$$h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot \hat{h}_t$$

A weighted average between old memory and the new candidate —  $z_t$  is the dial.

# Reset gate — how much past to use now

- Controls how much of the previous state feeds into the new candidate.
- $r_t$  close to 0  $\rightarrow$  ignore the past and act on the current input alone.
- Useful for dropping irrelevant history when a new pattern begins.
- Applied inside the candidate computation, before  $\tanh$ .

$$r_t = \sigma( W_r \cdot [h_{t-1}, x_t] + b_r )$$

## Candidate state

$$\hat{h}_t = \tanh( W \cdot [ r_t \odot h_{t-1}, x_t ] )$$

The reset gate  $r_t$  gates the previous state before it is mixed with the new input.

# All four equations together

UPDATE GATE

$$z_t = \sigma( Wz \cdot [h_{t-1}, x_t] )$$

RESET GATE

$$r_t = \sigma( Wr \cdot [h_{t-1}, x_t] )$$

CANDIDATE STATE

$$\hat{h}_t = \tanh( W \cdot [ r_t \odot h_{t-1}, x_t ] )$$

NEW HIDDEN STATE

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \hat{h}_t$$

*Four equations vs. LSTM's six — fewer gates, fewer parameters.*

05

# Comparison & Practice

Choosing and using the right model

# LSTM vs GRU at a glance

| Aspect            | LSTM                                     | GRU                             |
|-------------------|------------------------------------------|---------------------------------|
| Gates             | 3 (forget, input, output)                | 2 (update, reset)               |
| Memory            | Separate cell state $C_t$ + hidden $h_t$ | Single hidden state $h_t$       |
| Parameters        | More ( $\approx 4 \times$ RNN)           | Fewer ( $\approx 3 \times$ RNN) |
| Training speed    | Slower                                   | Faster                          |
| Long dependencies | Very strong                              | Strong                          |
| Best when         | Large data, very long sequences          | Less data, need speed           |

# RNN vs LSTM vs GRU

## Vanilla RNN

- Simplest, fastest
- No gating
- Fails on long sequences
- Vanishing gradients

## LSTM

- Cell state + 3 gates
- Best long-term memory
- Most parameters
- The proven workhorse

## GRU

- 2 gates, no cell state
- Fast & efficient
- Fewer parameters
- Great modern default



# Which one should you use?

1

## Start with GRU

A strong, efficient default — especially with limited data or compute.

2

## Reach for LSTM

When sequences are very long or you have lots of data and want maximum capacity.

3

## Vanilla RNN

Rarely in practice — mainly for teaching, tiny problems, or very short sequences.

4

## Beyond RNNs

For many tasks, Transformers now outperform recurrent nets — but RNN/GRU/LSTM remain vital for streaming and low-resource settings.

# Where these models are used

1

## Language modeling

Predicting the next word / character

2

## Machine translation

Seq-to-seq encoder-decoder models

3

## Speech recognition

Mapping audio frames to text

4

## Time-series forecasting

Demand, weather, finance, sensors

5

## Handwriting & OCR

Recognizing sequential strokes

6

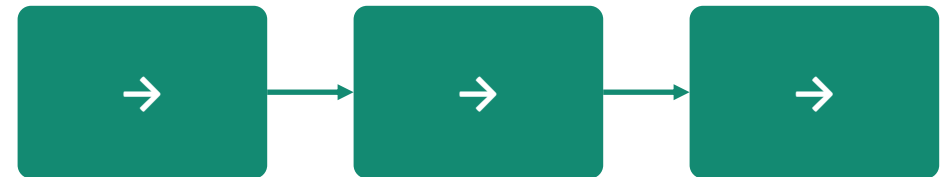
## Music & text generation

Producing sequences one step at a time

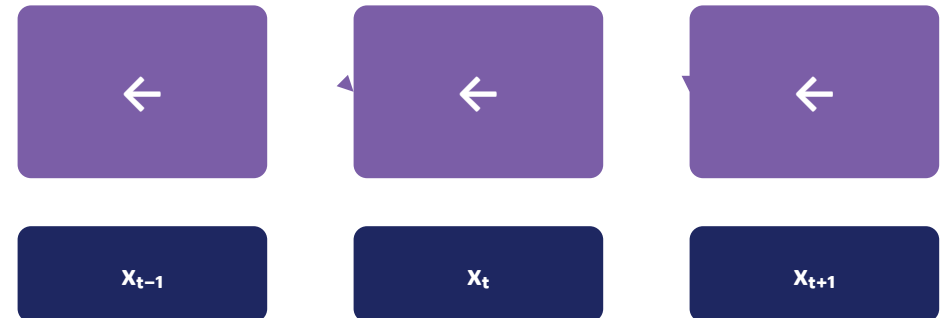
# Bidirectional RNNs

- Some tasks let you see the whole sequence at once (not streaming).
- A bidirectional RNN runs two passes: one left-to-right, one right-to-left.
- The two hidden states are concatenated, so each step sees past AND future context.
- Boosts accuracy for tagging, NER, and speech — anywhere both directions help.

forward pass

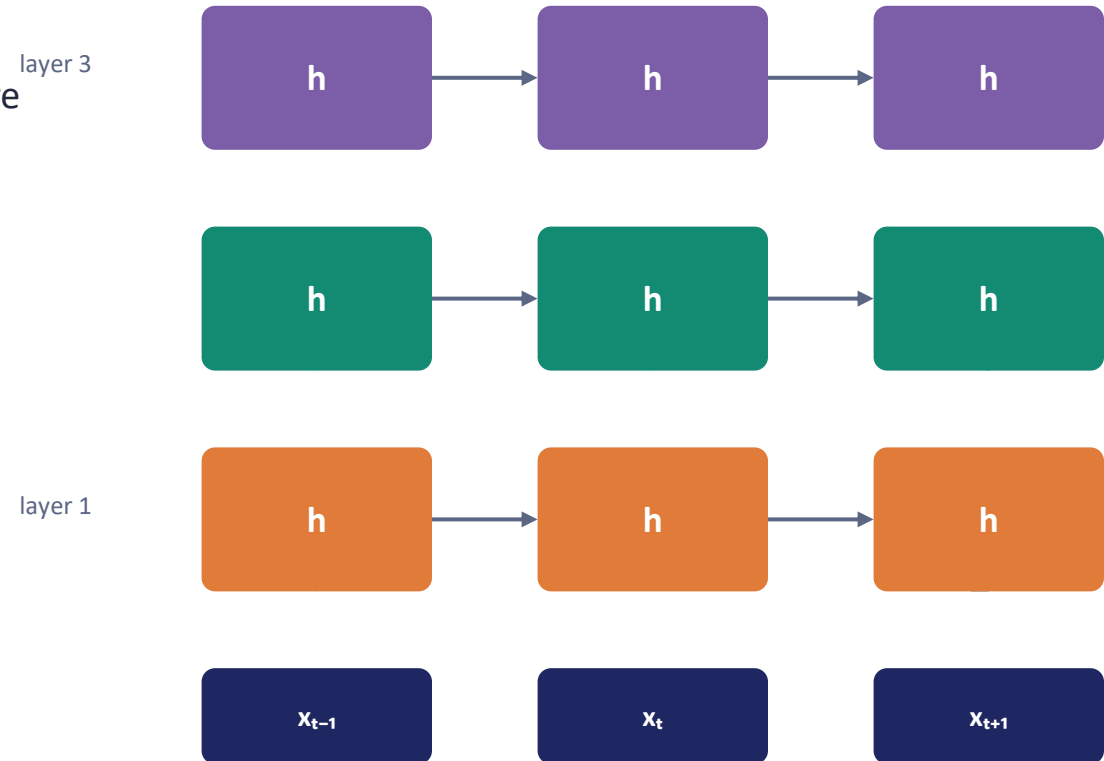


backward pass



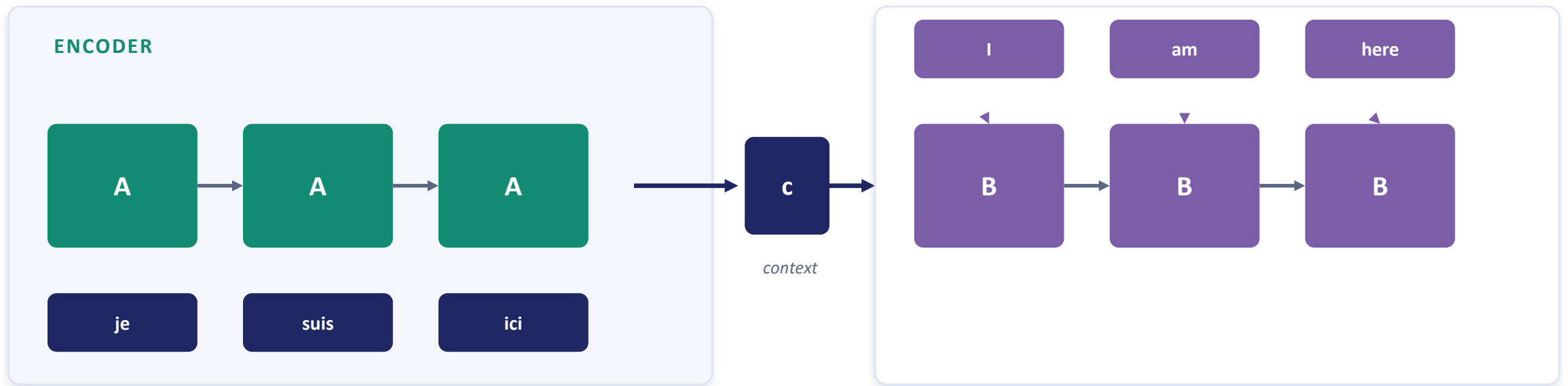
# Deep (stacked) RNNs

- Stack multiple recurrent layers so the output of one feeds the next.
- Lower layers capture short, local patterns; higher layers capture abstract structure.
- More capacity — at the cost of more parameters and harder training.
- Two to four layers is typical; add dropout between layers to regularize.



# Sequence-to-sequence (encoder-decoder)

The architecture behind translation and summarization — two RNNs joined by a context vector.



*The encoder compresses the input into a context vector  $c$ ; the decoder unrolls it into the output sequence.*

# Practical training tips

- Clip gradients (e.g. to a max norm of 5) to tame exploding gradients.
- Use a sensible sequence length; apply truncated BPTT for very long inputs.
- Stack layers or use bidirectional RNNs when future context is available.
- Add dropout between layers (not across time) for regularization.
- Normalize / batch your sequences with padding + masking for efficiency.
- Initialize forget-gate bias to 1 in LSTMs so the cell starts by remembering.
- Try both LSTM and GRU — the winner is task-dependent; validate empirically.

# Key takeaways

## RNNs add memory

A loop and shared weights let networks process sequences of any length.

## Gradients are the catch

Vanilla RNNs suffer vanishing/exploding gradients and forget long-range context.

## LSTM adds a memory highway

A cell state and three gates preserve information over long spans.

## GRU streamlines it

Two gates, one state — comparable accuracy with less compute.

## Choose empirically

GRU as a fast default, LSTM for capacity; validate on your own data.

# References

- Hochreiter & Schmidhuber (1997). Long Short-Term Memory. Neural Computation.
- Cho et al. (2014). Learning Phrase Representations using RNN Encoder–Decoder (GRU).
- Chung et al. (2014). Empirical Evaluation of Gated Recurrent Neural Networks.
- Christopher Olah — “Understanding LSTM Networks” (colah.github.io blog).
- Andrej Karpathy — “The Unreasonable Effectiveness of Recurrent Neural Networks”.
- Goodfellow, Bengio & Courville — Deep Learning (Ch. 10, Sequence Modeling).



# Thank you

Questions & discussion

RNN

LSTM

GRU